Kasi Yedumati · Full Stack Training

# Module 07 — Dictionaries

**Phase:** Core Python
**Previous:** Tuples and Sets
**Next:** Data Structures Overview

## Module Outcome

Create, update, search, and delete dictionary data, nest one level, and apply dictionaries to simple application-style problems.

By the end of this module you will be able to:

- Create a dict and look up values by key
- Choose `[]` vs `.get()` when a key might be missing
- Add, update, merge (`update`), and delete (`del` / `pop`)
- Loop with `.keys()`, `.values()`, and `.items()`
- Nest one level (address inside employee)
- Check membership (`in` checks keys) and avoid the shallow-copy trap

---

## 1. Why This Concept Matters

A list stores related facts by **position**. That works until someone has to remember what index 0 means:

```
student = ["Rahul", 22, "Hyderabad"]
print(student[0])   # name? age? city? — you must remember the order
```

We want one bag where each piece of information has a **name**:

```
name → Rahul
age  → 22
city → Hyderabad
```

That labelled bag is a **dictionary**. You look up a contact, a SKU, or a roll number by **key** — you do not walk the whole list.

A phone book is the same idea: you go to the name. You do not read every page.

---

## 2. Create and read

### Understand

A dictionary maps **key → value**.

- Keys must be **unique** and **hashable** (`str`, `int`, or a tuple of immutables — not a `list`).
- Values can be anything: a string, a number, a list, even another dict.
- Write a dict with `{ }`. Read a required field with `d["key"]`.

```
employee
  "emp_id"     →  101
  "name"       →  Ananya Sharma
  "department" →  Engineering
  "salary"     →  85000
```

### How It Works

```
employee = {
    "emp_id": 101,
    "name": "Ananya Sharma",
    "department": "Engineering",
    "salary": 85000,
}
print(employee["name"])
```

**Expected Output:**

```
Ananya Sharma
```

You may also build with `dict(...)` when the keys are simple names:

```
product = dict(sku="P001", name="Wireless Mouse", price=799)
print(product["price"])
```

**Expected Output:**

```
799
```

`{ }` is the everyday form.

### ⚠️ WATCH OUT

```
# Wrong — a list is not a valid key
{[1, 2]: "nope"}          # TypeError

# Duplicate keys: last value wins (no error)
{"a": 1, "a": 2}          # {"a": 2}
```

Prefer unique, hashable keys.

### Try It

Create a product dict with `sku`, `name`, and `price`. Print the name, then the price.

---

## 3. `.get()` vs `[]`

### Understand

Not every record has every field. A student may have no phone yet.

- `[]` is for a key that **must** exist. Missing → `KeyError`.
- `.get()` is for an **optional** field. Missing → `None`, or a default you choose.

### How It Works

| Style | Missing key |
|---|---|
| `d["email"]` | `KeyError` |
| `d.get("email")` | `None` |
| `d.get("email", "N/A")` | `"N/A"` |

```
student = {"name": "Priya", "roll": "2026001", "marks": 88}
print(student["name"])
print(student.get("phone", "N/A"))
print(student.get("email"))
```

**Expected Output:**

```
Priya
N/A
None
```

### ⚠️ WATCH OUT

```
# Wrong — assume every key exists
print(student["email"])     # KeyError

# Prefer when the field might be absent
print(student.get("email", "N/A"))
```

### Try It

Make `{"name": "Dev", "department": "Engineering"}`. Print `name` with `[]`. Print `email` with `.get(..., "Not set")`.

---

## 4. Add, update, and merge

### Understand

The same assignment does two jobs:

- Existing key → **update**
- New key → **add**

`update(other)` merges another dict. Overlapping keys are overwritten with **no warning**.

Contacts change numbers. Profiles gain an email and a city. Same syntax either way.

### How It Works

```
contact = {"name": "Dev Kumar", "phone": "9876543210"}
contact["phone"] = "9876500000"
contact["email"] = "dev@example.com"
contact["city"] = "Bengaluru"
print(contact)

contact.update({"role": "Intern", "company": "KSR Tech"})
print(contact)
```

**Expected Output:**

```
{'name': 'Dev Kumar', 'phone': '9876500000', 'email': 'dev@example.com', 'city': 'Bengaluru'}
{'name': 'Dev Kumar', 'phone': '9876500000', 'email': 'dev@example.com', 'city': 'Bengaluru',
'role': 'Intern', 'company': 'KSR Tech'}
```

### ⚠️ WATCH OUT

`update` overwrites overlapping keys without asking. If you must protect an existing value, check first: `if key in contact:`.

### Try It

Start with `{"name": "Ravi", "phone": "9876543210"}`. Change the phone, add `city`, then `update` with `{"role": "Student"}`. Print the dict.

---

## 5. Delete and `pop`

### Understand

Remove a field you no longer need — a wrong category on an expense, or a contact who left.

| Call | Effect |
|---|---|
| `del d["key"]` | Remove key (`KeyError` if missing) |
| `d.pop("key")` | Remove and **return** the value |
| `d.pop("key", default)` | Safe if missing |
| `d.clear()` | Empty the dict |

Use `pop` when you still need the removed value. Use `del` when you only need it gone.

### How It Works

```
expense = {"date": "01-08-2026", "category": "Food", "amount": 450}
amount = expense.pop("amount")
print(amount)
print(expense)

del expense["category"]
print(expense)
```

**Expected Output:**

```
450
{'date': '01-08-2026', 'category': 'Food'}
{'date': '01-08-2026'}
```

### ⚠️ WATCH OUT

```
# Wrong
d.pop("missing")            # KeyError

# Prefer when the key might be absent
d.pop("missing", None)
```

`del d["missing"]` raises `KeyError` the same way.

### Try It

From `{"item": "Chai", "qty": 2, "price": 20}`, `pop` `"price"`. Print the returned value, then print what remains.

---

## 6. Keys, values, and items

### Understand

Menus and printouts need every field without hard-coding each name.

- `.keys()` — the labels
- `.values()` — the data
- `.items()` — `(key, value)` pairs — the everyday report loop

These are **views** of the live dict. Wrap with `list(...)` when you need a snapshot.

Since Python 3.7, dictionaries keep **insertion order**. Still use a dict for **mapping**, not as a replacement for a list.

### How It Works

```
marks = {"Python": 88, "SQL": 76}
for subject, score in marks.items():
    print(f"{subject}: {score}")

product = {"sku": "P10", "name": "Chai Mug", "price": 299}
print(list(product.keys()))
print(list(product.values()))
```

**Expected Output:**

```
Python: 88
SQL: 76
['sku', 'name', 'price']
['P10', 'Chai Mug', 299]
```

### ⚠ WATCH OUT

Do not add or remove keys while looping the **same** dict. If you must change it, loop a snapshot: `for k, v in list(d.items()):`.

### Try It

Loop `{"Chai": 20, "Samosa": 25}` with `.items()` and print each line as `item: ₹price`.

---

## 7. Nested dict — one level

### Understand

A value can be another dict (or a list). One nesting level models most student, employee, and order shapes:

```
employee
  "name"    →  Meera
  "address" →  { "city" → Pune,  "pincode" → 411001 }
  "skills"  →  ["Python", "SQL"]
```

Read nested data with two lookups: `employee["address"]["city"]`.

### How It Works

```
employee = {
    "name": "Meera",
    "address": {"city": "Pune", "pincode": "411001"},
    "skills": ["Python", "SQL"],
}
print(employee["address"]["city"])
print(employee["skills"][0])

employee["address"]["city"] = "Bengaluru"
print(employee["address"])
```

**Expected Output:**

```
Pune
Python
{'city': 'Bengaluru', 'pincode': '411001'}
```

### ⚠ WATCH OUT

`employee.copy()` does **not** deep-copy nested dicts. Changing a nested field on the copy can change the original too. Update nested fields on purpose. Full cloning with `copy.deepcopy` is an interview topic — not required for the lab.

### Try It

Create `{"name": "Ravi", "address": {"city": "Hyderabad", "pincode": "500001"}}`. Print the city. Change pincode to `"500081"` and print the address dict.

---

## 8. Membership and copy

### Understand

`"phone" in contact` checks **keys**, not values.

Lists already taught this: `backup = user` is an **alias** (same object, two names). `user.copy()` is a **shallow** copy — top-level keys are separate; nested objects may still be shared.

Menus check “does this name exist?” before update. A backup must not silently rewrite the live contact.

### How It Works

```
contact = {"name": "Ravi", "phone": "9876543210"}
print("Found" if "email" in contact else "Missing")
print("Ravi" in contact)
print("Ravi" in contact.values())

user = {"username": "aisha", "role": "staff"}
backup = user.copy()
backup["role"] = "admin"
print(user["role"])
print(backup["role"])
```

**Expected Output:**

```
Missing
False
True
staff
admin
```

Flat copies are independent. Nested dicts are not:

```
original = {"name": "Meera", "address": {"city": "Pune"}}
snapshot = original.copy()
snapshot["address"]["city"] = "Bengaluru"
print(original["address"]["city"])
```

**Expected Output:**

```
Bengaluru
```

`.copy()` is enough for a **flat** backup (name → phone). Nested trees share the inner dict unless you deep-copy.

### ⚠ WATCH OUT

```
# Wrong — thinking `in` searches values
"Ravi" in contact           # False  (Ravi is a value, not a key)

# Prefer
"Ravi" in contact.values()
```

### Try It

Given `{"name": "Priya", "phone": "9123456780"}`, print whether `"email"` is in the dict. Then print whether `"Priya"` is in `.values()`.

---

## Step-by-Step: Build It

One assemble: seed two contacts (name → phone), search safely, list with `.items()`, update, add, and delete — the menu skills for tonight's kit.

```
contacts = {
    "Ravi": "9876543210",
    "Priya": "9123456780",
}

# Safe search
print(contacts.get("Ravi", "Not found"))
print(contacts.get("Dev", "Not found"))

# List
for name, phone in contacts.items():
    print(f"{name}: {phone}")

# Update + add
contacts["Ravi"] = "9876500000"
contacts["Dev"] = "9000000001"

# Delete safely
removed = contacts.pop("Priya", None)
print("Removed:", removed)
print("Left:", contacts)
```

**Expected Output:**

```
9876543210
Not found
Ravi: 9876543210
Priya: 9123456780
Removed: 9123456780
Left: {'Ravi': '9876500000', 'Dev': '9000000001'}
```

Using the same key twice overwrites the previous contact. Check `if name in contacts:` before add if you want "already exists" behaviour.

---

## Practice Exercises

Attempt each problem before reading the model answer.

1. Course hours `{"Python": 40, "SQL": 20, "Git": 10}`. Print Python hours with `[]`. Print `"Django"` hours with `.get(..., 0)`.

**Hint:** Required key → `[]`. Missing key → `.get`.

**Model answer:**

```
hours = {"Python": 40, "SQL": 20, "Git": 10}
print(hours["Python"])
print(hours.get("Django", 0))
```

**Expected Output:**

```
40
0
```

2. Start with `{"id": "T-12", "status": "Open"}`. Change status to `"Closed"`, add `"agent": "Meera"`, print the dict.

**Model answer:**

```
ticket = {"id": "T-12", "status": "Open"}
ticket["status"] = "Closed"
ticket["agent"] = "Meera"
print(ticket)
```

**Expected Output:**

```
{'id': 'T-12', 'status': 'Closed', 'agent': 'Meera'}
```

3. From `{"Pen": 10, "Notebook": 40, "USB": 199}`, `pop` `"USB"` and print the returned price and what remains. Then `pop("Mug", 0)` and print that result.

**Hint:** The second `pop` must use a default so a missing key does not crash.

**Model answer:**

```
prices = {"Pen": 10, "Notebook": 40, "USB": 199}
usb = prices.pop("USB")
print(usb)
print(prices)
print(prices.pop("Mug", 0))
```

**Expected Output:**

```
199
{'Pen': 10, 'Notebook': 40}
0
```

4. Loop `{"Priya": 88, "Ravi": 76, "Dev": 95}` with `.items()` and print only names whose marks are **≥ 80**.

**Hint:** Combine `.items()` with `if` — same idea as filtering a list.

**Model answer:**

```
marks = {"Priya": 88, "Ravi": 76, "Dev": 95}
for name, score in marks.items():
```

```
    if score >= 80:
        print(name, score)
```

**Expected Output:**

```
Priya 88
Dev 95
```

5. Nested order:

```
order = {
    "id": "ORD-1",
    "customer": {"name": "Meera", "city": "Pune"},
    "total": 499,
}
```

Print the customer name. Change city to `"Hyderabad"`. Add `"status": "Paid"` at the top level. Print `order["customer"]` and `order["status"]`.

**Model answer:**

```
order = {
    "id": "ORD-1",
    "customer": {"name": "Meera", "city": "Pune"},
    "total": 499,
}
print(order["customer"]["name"])
order["customer"]["city"] = "Hyderabad"
order["status"] = "Paid"
print(order["customer"])
print(order["status"])
```

**Expected Output:**

```
Meera
{'name': 'Meera', 'city': 'Hyderabad'}
Paid
```

---

## Lab

Combine create, `.get()`, nested values, update, add, list, and safe delete. No `def`, no files, no JSON.

### Problem

A trainer keeps batch records as **roll → profile**. Lookup must not crash when a roll is missing.

### What to Build

A short script that seeds two students, looks up, updates, adds, lists, and deletes.

### Requirements

- `students = {}` — roll string → `{"name": ..., "marks": ...}`

- Seed two rolls
- Lookup an existing roll and print the **name**
- Lookup a missing roll with `.get` and print `"Not found"`
- Update one student's marks
- Add a third student
- Delete one roll with `pop(..., None)` and print `Removed: <name>`
- List what remains as `roll: name — marks`

### Expected Behavior

Missing lookup prints a message — never a `KeyError`. After delete, the listed rolls are only the ones still in the dict.

### Sample Run

```
Priya
Not found
Removed: Priya
2026002: Ravi — 81
2026003: Dev — 95
```

(Your seed data may differ; the **shape** of the output should match.)

### Constraints

Top-level statements only — no `def`, no type hints, no `json`.

### Bonus

Print the name of whoever has the highest marks (loop `.items()`, track the max).

---

## Mini-Project (Assignment): Contact Book CLI

**Why this matters:** A contacts app is a dictionary: name → phone. Add, lookup, list, and delete are the same operations every CRUD screen uses — employee directories, product catalogs, ticket stores.

**Lab kit:** 15_Lab_Assets/11_contact_book/

| File | What you do |
|---|---|
| `starter/contact_book.py` | Build the `while True` menu. **No `def` required.** Prefer `name.title()` as the key |
| `broken/contact_book.py` | `KeyError` on missing lookup — fix with `.get` / `in` |
| `solution/` | Do **not** copy this |

This is **not** the Step-by-Step assemble from class. Same dict tools, full menu ticket.

### Objective

Store contacts as **name → phone** and drive add, lookup, delete, and list from a menu. Missing names must not crash.

## Story / Scenario

Placement cell keeps student phone numbers. Staff type a name and expect the number — or a clear “not found”, never a traceback.

## Requirements

| Piece | Detail |
|---|---|
| Data | Single `contacts = {}` — name → phone string |
| Menu | **1** Add · **2** Lookup · **3** Delete · **4** List · **5** Quit |
| Safe lookup | `.get` or `in` — never dump a traceback on a missing name |
| Add | Warn if the name already exists (optional but good) |
| Style | Top-level statements only — no `def`, no type hints, no `json` yet |

## Expected Behavior

- Add stores `name.title()` → phone
- Lookup of a missing name prints a message
- List prints every `name: phone`
- Quit exits cleanly

## Sample Run

```
1 Add  2 Lookup  3 Delete  4 List  5 Quit
Choice: 1
Name: ravi
Phone: 9876543210
Added.
Choice: 2
Name: Ravi
9876543210
Choice: 4
Ravi: 9876543210
Choice: 5
Bye.
```

## Bonus Challenges

- Search by phone (scan `.items()` / `.values()`) when the name is unknown
- Update phone for an existing name
- **Later — Files / JSON week:** save and load `contacts.json`

## Done when

- [ ] Lookup missing name does not crash (use `.get` or `in`)
- [ ] Add / delete / list work from the menu
- [ ] Quit exits cleanly

- [ ] No `def` / type hints / JSON on the core path

## Interview Q&A

**Q1. Dict vs list for lookup by ID?**
A dict finds the value by key (average O(1)). A list must scan row by row (O(n)). Use a dict when you have a natural key — `emp_id`, SKU, name.

**Q2. `.get` vs `[]`?**
`[]` raises `KeyError` if the key is missing. `.get(key, default)` returns the default (`None` if you omit it). Required field → `[]`. Optional field → `.get`.

**Q3. Are dict keys ordered?**
Insertion order is preserved since Python 3.7. Still use a dict for **mapping**, not as a primary ordered list.

**Q4. What does `"x" in d` check?**
Keys only — not values. Use `"x" in d.values()` when you mean values.

**Q5. What is a shallow copy?**
`.copy()` copies top-level pairs. Nested mutable values may still be shared. Use `copy.deepcopy` for a nested tree when you truly need a full clone.

**Q6. Can values be lists or dicts?**
Yes. An order can hold `"items": [{"sku": "P1", "qty": 2}]`. An employee can hold `"address": {"city": "Pune"}`.

**Q7. Why prefer a dict over a list of pairs for contacts?**
Name → phone is a natural key lookup. You avoid scanning every row, and the code reads as “get this name.”

**Q8. How do APIs often ship this shape?**
JSON objects become Python dicts. `json.dumps` / `json.loads` come in Files week — not required for this lab.

## Cheat Sheet

```
{}                  → create a dict
d["k"]              → read / write (KeyError if missing on read)
d.get("k", default) → safe read
d["k"] = v          → add or update
d.update(other)     → merge (overwrites overlap)
del d["k"]          → remove (KeyError if missing)
d.pop("k", default) → remove and return
d.clear()           → empty the dict
d.keys()            → key view
d.values()          → value view
d.items()           → (key, value) view — everyday loop
"k" in d            → key membership (not values)
d.copy()            → shallow copy (nested objects may be shared)
```

Keys must be unique and hashable. Duplicate keys in a literal: last value wins.